JAVA / 阶段 02

面向对象程序设计

阶段二 · 答案

对应原题库全部 17 道题

完整代码 / 输入输出 / 基础思路

使用方式

每道题保存为独立的 Main.java。先读题并自己尝试，再对照答案运行；不要把不同题的 Main 类放进同一个文件。

命令：javac -encoding UTF-8 Main.java；随后执行 java Main。输入样例中的数据后按回车。网站可直接编辑、编译和运行。

解法使用变量、判断、循环、数组、方法和基础面向对象知识，不依赖第三方 Java 库。

题目口径与说明

1. 依据用户提供的阶段二题库 PDF 整理，共17道独立编程题。原题前7题后又重复编号6、7，本资料按出现顺序编号1至17，并保留原题号。

2. 采用中国大学二年级常见基础写法：普通类、字段、构造、getter/setter、if、for、数组、继承、接口与抽象类；不使用Stream、Lambda或第三方库。

3. 每题均有独立public class Main；其余顶层类和接口不加public，保存为Main.java即可。不同题中的同名类必须分开编译运行。

4. 图形绘制题和狮子题已对照原 PDF 类图及运行截图，字段、参数与继承关系按原图整理。

5. 明确约定：Point返回距离平方；同时提供无参/有参构造；银行账户无透支，最终余额1500元；Vehicle的start/stop按文字要求各带一个String参数。

6. Circle/Cylinder区分底面积与表面积，并使用super.getArea防止多态导致体积公式错误；CanCry保留G类及G.main，由Main启动。

目录

第 01 题 三维点 Point 与到原点的距离平方 3

第 02 题 长方形的面积、周长与 toString 4

第 03 题 学生类的访问控制与两种构造 5

第 04 题 银行账户存款、取款与余额检查 6

第 05 题 手机类与两种对象初始化方式 8

第 06 题 学生阅读图书：对象作为参数 9

第 07 题 Person：姓名校验、固定血型与身高不减 10

第 08 题 People 基类、Employee 与 Teacher 子类 12

第 09 题 Animal、Cat、Bird 与运行时多态 14

第 10 题 Vehicle 接口、Bike/Bus 与 Driver 15

第 11 题 Circle 与 Cylinder：继承、面积重写和体积 17

第 12 题 CanCry 接口与 G.makeCry 测试 18

第 13 题 抽象 Shape 与圆、矩形面积 20

第 14 题 员工数组、平均工资项目与应发工资 21

第 15 题 Graph 类图：三种图形与 Client 绘制 23

第 16 题 Animal、Ibehavior 与 Lion 类图实现 24

第 17 题 网络用户类与默认邮箱构造重载 26

复习与参考 27

第 01 题 三维点 Point 与到原点的距离平方

原题编号：1；本书按出现顺序编为第 1 题。

题目

定义 Point 类，用浮点型成员 x、y、z 表示三维坐标。提供构造方法、setX、setY、setZ，getDistance() 返回点到原点的距离平方。原题写 Point() 又要求特定坐标，因此同时提供无参构造 Point() 和有参构造 Point(double x, double y, double z)，并实例化验证。

基础思路

用 double 保存坐标。无参构造创建原点，有参构造直接接收坐标；三个 setter 分别改坐标，距离平方为 x*x+y*y+z*z。

完整参考代码 · Main.java

```
class Point {
    private double x;
    private double y;
    private double z;

    public Point() {
        this(0, 0, 0);
    }

    public Point(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    public void setX(double x) { this.x = x; }
    public void setY(double y) { this.y = y; }
    public void setZ(double z) { this.z = z; }

    public double getDistance() {
        return x * x + y * y + z * z;
    }
}

public class Main {
    public static void main(String[] args) {
        Point p1 = new Point();
        System.out.println("原点的距离平方：" + p1.getDistance());
        Point p2 = new Point(1, 2, 3);
        System.out.println("点(1,2,3)的距离平方：" + p2.getDistance());
        p1.setX(3);
        p1.setY(4);
        p1.setZ(0);
        System.out.println("点(3,4,0)的距离平方：" + p1.getDistance());
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
原点的距离平方：0.0
点(1,2,3)的距离平方：14.0
点(3,4,0)的距离平方：25.0
```

注意

Java 的 ^ 是按位异或，不是乘方；平方直接写 x * x。

第 02 题 长方形的面积、周长与 toString

原题编号：2；本书按出现顺序编为第 2 题。

题目

设计长方形类，成员为长 length、宽 width；构造方法初始化两个字段；getArea() 求面积，getPerimeter() 求周长；重写 Object.toString()。测试类从标准输入依次读取长、宽并创建对象，通过 toString 输出长、宽、面积、周长。输入 10 和 20 时输出“这个长方形长为10，宽为20，面积为200，周长为60。”。本基础示例按原示例采用正整数边长。

基础思路

用 int 保存示例中的整数边长，面积为 length*width，周长为 2*(length+width)，把完整说明放入 toString。

完整参考代码 · Main.java

```
import java.util.Scanner;

class Rectangle {
    private int length;
    private int width;

    public Rectangle(int length, int width) {
        this.length = length;
        this.width = width;
    }

    public int getArea() { return length * width; }
    public int getPerimeter() { return 2 * (length + width); }

    @Override
    public String toString() {
        return "这个长方形长为" + length + "，宽为" + width
            + "，面积为" + getArea() + "，周长为" + getPerimeter() + "。";
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int length = sc.nextInt();
        int width = sc.nextInt();
        Rectangle r = new Rectangle(length, width);
```

Main.java（续）

```
        System.out.println(r.toString());
    }
}
```

样例输入

```
10
20
```

样例输出

```
这个长方形长为10，宽为20，面积为200，周长为60。
```

注意

toString 名称的 S 必须大写，不能写 toString(String s)。

第 03 题 学生类的访问控制与两种构造

原题编号：3；本书按出现顺序编为第 3 题。

题目

设计 Student 类，保存姓名、年龄、身高、年级：姓名为 public 字段，其他字段为 private；年龄为读写属性，使用 getAge()/setAge() 实现。提供无参和全参构造方法并实例化测试。为展示私有字段，增加 showInfo() 方法。

基础思路

按要求区分字段权限，用 getter 和 setter 公开年龄读写；通过两个构造方法展示默认对象和完整初始化对象。

完整参考代码 · Main.java

```
class Student {
    public String name;
    private int age;
    private double height;
    private String grade;

    public Student() {
        this("未命名", 18, 1.60, "大一");
    }

    public Student(String name, int age, double height, String grade) {
        this.name = name;
        this.age = age;
        this.height = height;
        this.grade = grade;
    }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
```

Main.java（续）

```
    public void showInfo() {
        System.out.println("姓名：" + name + "，年龄：" + age
            + "，身高：" + height + "米，年级：" + grade);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "张三";
        s1.setAge(19);
        s1.showInfo();
        Student s2 = new Student("李四", 20, 1.75, "大二");
        s2.showInfo();
        System.out.println("李四的年龄：" + s2.getAge());
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
姓名：张三，年龄：19，身高：1.6米，年级：大一
姓名：李四，年龄：20，身高：1.75米，年级：大二
李四的年龄：20
```

注意

不要在 Student 外直接写 s1.age。

第 04 题 银行账户存款、取款与余额检查

原题编号：4；本书按出现顺序编为第 4 题。

题目

创建 BankAccount 类，私有字段为账号和余额，为两字段提供 setter/getter，提供全参构造和存款、取款、查询余额方法。在 main 创建 ba，账号 123456、余额 500 元，存入 1000 元，尝试取出 2000 元，最后查询余额。这里采用普通无透支账户规则：余额不足时拒绝取款，因此最终余额 1500 元。

基础思路

账号用 String，余额用 double。操作先验证金额，再判断余额；取款失败时 return，不改变原余额。

完整参考代码 · Main.java

```
class BankAccount {
    private String account;
    private double balance;
```

Main.java（续）

```
    public BankAccount(String account, double balance) {
        setAccount(account);
        setBalance(balance);
    }

    public String getAccount() { return account; }
    public void setAccount(String account) { this.account = account; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) {
        if (balance < 0) {
            throw new IllegalArgumentException("余额不能为负数");
        }
        this.balance = balance;
    }

    public void deposit(double money) {
        if (money <= 0) {
            System.out.println("存款金额必须大于0");
            return;
        }
        balance += money;
        System.out.println("存款成功：" + money + "元");
    }

    public void withdraw(double money) {
        if (money <= 0) {
            System.out.println("取款金额必须大于0");
            return;
        }
        if (money > balance) {
            System.out.println("余额不足，取款失败");
            return;
        }
        balance -= money;
        System.out.println("取款成功：" + money + "元");
    }

    public void queryBalance() {
        System.out.println("账号：" + account + "，余额：" + balance + "元");
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount ba = new BankAccount("123456", 500);
        ba.deposit(1000);
        ba.withdraw(2000);
        ba.queryBalance();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
存款成功：1000.0元
余额不足，取款失败
账号：123456，余额：1500.0元
```

注意

只输出“余额不足”却继续执行减法仍会造成负余额，必须 return 或使用 else。

第 05 题 手机类与两种对象初始化方式

原题编号：5；本书按出现顺序编为第 5 题。

题目

创建 Phone 类，私有字段为品牌和价格，提供两字段的 setter/getter、无参和全参构造，以及打电话、听歌两个方法。在 main 中分别用 setter 和全参构造创建两部手机，输出“正在使用价格为998元的华为手机打电话”和“正在使用价格为998元的小米手机听歌”。空格仅为排版，本答案使用无空格的中文行。

基础思路

先 new Phone() 再设置华为手机信息；小米手机直接调用全参构造。行为方法读取各自对象的品牌和价格。

完整参考代码 · Main.java

```
class Phone {
    private String brand;
    private int price;

    public Phone() { }

    public Phone(String brand, int price) {
        this.brand = brand;
        this.price = price;
    }

    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }

    public void call() {
        System.out.println("正在使用价格为" + price + "元的" + brand + "手机打电话");
    }

    public void listenMusic() {
        System.out.println("正在使用价格为" + price + "元的" + brand + "手机听歌");
    }
}

public class Main {
    public static void main(String[] args) {
        Phone p1 = new Phone();
        p1.setBrand("华为");
```

Main.java（续）

```
        p1.setPrice(998);
        Phone p2 = new Phone("小米", 998);
        p1.call();
        p2.listenMusic();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
正在使用价格为998元的华为手机打电话
正在使用价格为998元的小米手机听歌
```

注意

有参构造存在时不会自动生成无参构造，需要自己声明 Phone()。

第 06 题 学生阅读图书：对象作为参数

原题编号：6；本书按出现顺序编为第 6 题。

题目

创建 Book 类，私有属性为作者和书名；创建 Student 类，私有属性为姓名和年龄，提供一个读书方法，所读图书由参数传入。两类均提供所有属性的 setter/getter 和全参构造。在 main 创建一名18岁的张三和《资治通鉴》图书对象，调用读书方法，输出“18岁的张三正在读的书是《资治通鉴》”。

基础思路

Book 单独管理图书信息；Student.readBook(Book book) 通过传入图书的 getTitle() 获取书名并打印。

完整参考代码 · Main.java

```
class Book {
    private String author;
    private String title;

    public Book(String author, String title) {
        this.author = author;
        this.title = title;
    }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
}

class Student {
```

Main.java（续）

```
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    public void readBook(Book book) {
        System.out.println(age + "岁的" + name + "正在读的书是《"
            + book.getTitle() + "》");
    }
}

public class Main {
    public static void main(String[] args) {
        Book book = new Book("司马光", "资治通鉴");
        Student student = new Student("张三", 18);
        student.readBook(book);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
18岁的张三正在读的书是《资治通鉴》
```

注意

Student 不能直接访问 Book 的 private title，应调用 getTitle()。

第 07 题 Person：姓名校验、固定血型与身高不减

原题编号：7；本书按出现顺序编为第 7 题。

题目

设计 Person 类：姓名 private，血型 private 且创建后不可修改，另有身高字段。提供构造方法、getName、setName 和 addHeight；姓名必须为2～4个字符，身高不能减少。main 创建对象，并验证正常改名、非法改名、增加身高与负增长被拒绝。为读取结果补充 getBloodType 和 getHeight。

基础思路

血型声明 private final 且只在构造中赋值，不提供 setter。姓名统一在 setName 校验，addHeight 只接受非负增量。

完整参考代码 · Main.java

```
class Person {
    private String name;
    private final String bloodType;
    private double height;

    public Person(String name, String bloodType, double height) {
        if (height < 0) {
            throw new IllegalArgumentException("身高不能为负数");
        }
        setName(name);
        this.bloodType = bloodType;
        this.height = height;
    }

    public String getName() { return name; }
    public String getBloodType() { return bloodType; }
    public double getHeight() { return height; }

    public void setName(String name) {
        if (name == null || name.length() < 2 || name.length() > 4) {
            throw new IllegalArgumentException("姓名必须为2～4个字符");
        }
        this.name = name;
    }

    public void addHeight(double increase) {
        if (increase < 0) {
            throw new IllegalArgumentException("身高不能减少");
        }
        height += increase;
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person("张三", "A", 170);
        System.out.println("姓名：" + p.getName() + "，血型：" + p.getBloodType()
            + "，身高：" + p.getHeight() + "厘米");
        p.setName("李小明");
        p.addHeight(2);
        System.out.println("修改后：" + p.getName() + "，身高：" + p.getHeight() + "厘米");
        try {
            p.setName("王");
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        try {
            p.setName("一二三四五");
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        try {
            p.addHeight(-5);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("最终：" + p.getName() + "，血型：" + p.getBloodType()
```

Main.java（续）

```
            + "，身高：" + p.getHeight() + "厘米");
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
姓名：张三，血型：A，身高：170.0厘米
修改后：李小明，身高：172.0厘米
姓名必须为2～4个字符
姓名必须为2～4个字符
身高不能减少
最终：李小明，血型：A，身高：172.0厘米
```

注意

只不写血型 setter 不够直观，使用 private final 明确不可修改。

第 08 题 People 基类、Employee 与 Teacher 子类

原题编号：6（第二组）；本书按出现顺序编为第 8 题。

题目

原题第二次编号6：写 People 类作为基类，包含 protected String name、protected int age、People(String name,int age) 构造和 public getAge()。Employee 继承 People，新增 protected String empno；Teacher 继承 People，新增 String 类型 teano、grade。为子类编写构造并实例化测试。本答案把教师新增字段同样声明为 protected，便于展示继承结构。

基础思路

父类保存姓名和年龄。子类构造先 super(name,age) 初始化父类部分，再设置自己的编号、年级。

完整参考代码 · Main.java

```
class People {
    protected String name;
    protected int age;

    public People(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public int getAge() { return age; }
}

class Employee extends People {
```

Main.java（续）

```
    protected String empno;

    public Employee(String name, int age, String empno) {
        super(name, age);
        this.empno = empno;
    }

    public void showInfo() {
        System.out.println("员工：" + name + "，年龄：" + getAge() + "，工号：" + empno);
    }
}

class Teacher extends People {
    protected String teano;
    protected String grade;

    public Teacher(String name, int age, String teano, String grade) {
        super(name, age);
        this.teano = teano;
        this.grade = grade;
    }

    public void showInfo() {
        System.out.println("教师：" + name + "，年龄：" + getAge()
            + "，教师号：" + teano + "，年级：" + grade);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee("张三", 25, "E001");
        Teacher t = new Teacher("李四", 35, "T001", "大二");
        e.showInfo();
        t.showInfo();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
员工：张三，年龄：25，工号：E001
教师：李四，年龄：35，教师号：T001，年级：大二
```

注意

不能写 class Employee implements People，类继承类要用 extends。

第 09 题 Animal、Cat、Bird 与运行时多态

原题编号：7（第二组）；本书按出现顺序编为第 9 题。

题目

原题第二次编号7：定义 Animal，包含 name、构造和 sing()；Cat 继承它，新增 furColor、构造并重写 sing()；Bird 继承它，新增 featherColor、构造并重写 sing()。main 输出“1.Cat 2.Bird”，从标准输入读取1或2，创建相应对象并赋给 Animal 类型变量 pet，最后调用 pet.sing()。原题未指定名字和颜色输入，因此示例固定名字、颜色，仅由用户选择种类。

基础思路

变量声明为 Animal，按输入建立 Cat 或 Bird，再统一调用 sing，Java 会执行实际对象所属子类的方法。

完整参考代码 · Main.java

```
import java.util.Scanner;

class Animal {
    protected String name;

    public Animal(String name) { this.name = name; }

    public void sing() {
        System.out.println(name + "在叫");
    }
}

class Cat extends Animal {
    private String furColor;

    public Cat(String name, String furColor) {
        super(name);
        this.furColor = furColor;
    }

    @Override
    public void sing() {
        System.out.println("猫" + name + "，毛色：" + furColor + "，喵喵喵");
    }
}

class Bird extends Animal {
    private String featherColor;

    public Bird(String name, String featherColor) {
        super(name);
        this.featherColor = featherColor;
    }

    @Override
    public void sing() {
        System.out.println("鸟" + name + "，羽毛颜色：" + featherColor + "，叽叽喳喳");
    }
}
```

Main.java（续）

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("1.Cat  2.Bird");
        int choice = sc.nextInt();
        Animal pet;
        if (choice == 1) {
            pet = new Cat("小花", "白色");
        } else if (choice == 2) {
            pet = new Bird("小黄", "黄色");
        } else {
            System.out.println("请选择1或2");
            return;
        }
        pet.sing();
    }
}
```

样例输入

```
1
```

样例输出

```
1.Cat  2.Bird
猫小花，毛色：白色，喵喵喵
```

注意

pet 的声明类型虽然是 Animal，也不表示一定执行父类 sing。

第 10 题 Vehicle 接口、Bike/Bus 与 Driver

原题编号：8；本书按出现顺序编为第 10 题。

题目

创建 Vehicle 接口，声明 start、stop 两个“各带一个参数”的方法；Bike、Bus 实现接口。Driver 类定义 void drive(Vehicle v)，main 创建 Bike、Bus、Driver，分别传入 drive。原文同时把方法写作 start()/stop()，与“一个参数”矛盾；本答案优先满足文字要求，统一声明 start(String driverName)、stop(String driverName)，参数表示驾驶者，Driver.drive(Vehicle v) 的签名保持原要求。

基础思路

接口规定启动和停止的共同调用方式；两种交通工具分别实现，Driver 只接收 Vehicle 并依次调用两个行为。

完整参考代码 · Main.java

```
interface Vehicle {
    void start(String driverName);
    void stop(String driverName);
}

class Bike implements Vehicle {
    @Override
    public void start(String driverName) {
        System.out.println(driverName + "启动自行车");
    }

    @Override
    public void stop(String driverName) {
        System.out.println(driverName + "停止自行车");
    }
}

class Bus implements Vehicle {
    @Override
    public void start(String driverName) {
        System.out.println(driverName + "启动公交车");
    }

    @Override
    public void stop(String driverName) {
        System.out.println(driverName + "停止公交车");
    }
}

class Driver {
    public void drive(Vehicle v) {
        v.start("司机");
        v.stop("司机");
    }
}

public class Main {
    public static void main(String[] args) {
        Bike bike = new Bike();
        Bus bus = new Bus();
        Driver driver = new Driver();
        driver.drive(bike);
        driver.drive(bus);
    }
}
```

样例输入

（无输入，直接运行）

样例输出

```
司机启动自行车
司机停止自行车
司机启动公交车
司机停止公交车
```

注意

实现接口用 implements，不能写 extends Vehicle 来声明普通实现类。

第 11 题 Circle 与 Cylinder：继承、面积重写和体积

原题编号：9；本书按出现顺序编为第 11 题。

题目

Circle 包含私有 double Radius，无参构造将半径设为0，有参 Circle(double r) 设置半径；提供 getArea 和 getPerimeter。Cylinder 继承 Circle，增加私有 double height，通过 Cylinder(double r,double h) 设置半径与高，提供 getVolume 并重写 getArea 求圆柱表面积。main 创建半径3、高5的圆柱，分别显示底面圆面积、圆周长、圆柱体积和表面积。原题未指定输入，本例用构造参数设置。

基础思路

圆面积 πr^2，周长$2\pi r$；圆柱体积=底面积×高，表面积=2×底面积+周长×高。子类用 super.getArea() 取得底面积，避免把重写后的表面积误当底面积。

完整参考代码 · Main.java

```
import java.util.Locale;

class Circle {
    private double Radius;

    public Circle() { this(0); }

    public Circle(double r) {
        this.Radius = r;
    }

    public double getArea() {
        return Math.PI * Radius * Radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * Radius;
    }
}

class Cylinder extends Circle {
    private double height;

    public Cylinder(double r, double h) {
        super(r);
        this.height = h;
    }

    public double getBaseArea() {
        return super.getArea();
    }

    public double getVolume() {
        return super.getArea() * height;
```

Main.java（续）

```
    }

    @Override
    public double getArea() {
        return 2 * super.getArea() + super.getPerimeter() * height;
    }
}

public class Main {
    public static void main(String[] args) {
        Cylinder c = new Cylinder(3, 5);
        System.out.printf(Locale.ROOT, "圆面积：%.2f%n", c.getBaseArea());
        System.out.printf(Locale.ROOT, "圆周长：%.2f%n", c.getPerimeter());
        System.out.printf(Locale.ROOT, "圆柱体积：%.2f%n", c.getVolume());
        System.out.printf(Locale.ROOT, "圆柱表面积：%.2f%n", c.getArea());
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
圆面积：28.27
圆周长：18.85
圆柱体积：141.37
圆柱表面积：150.80
```

注意

getVolume 不能使用重写后的 getArea 当底面积，否则得到表面积乘高度。

第 12 题 CanCry 接口与 G.makeCry 测试

原题编号：10；本书按出现顺序编为第 12 题。

题目

定义接口 CanCry，包含 public void cry()；Dog、Cat 实现接口，分别准确输出“我是狗，我的叫声是汪汪汪”“我是猫，我的叫声是喵喵喵”。定义 G 类和 void makeCry(CanCry c)，在 G.main 内创建 dog、cat、g 并调用 g.makeCry。原题 makecry 与 makeCry 大小写不一致，这里统一为 makeCry。为在线练习统一 Main.java，额外增加 public Main 作为入口，委托给保留的 G.main。

基础思路

CanCry 统一叫声接口，makeCry 只调用 c.cry。保留题面 G 类与测试对象，Main 只负责启动 G.main。

完整参考代码 · Main.java

```
interface CanCry {
    public void cry();
}

class Dog implements CanCry {
    @Override
    public void cry() {
        System.out.println("我是狗，我的叫声是汪汪汪");
    }
}

class Cat implements CanCry {
    @Override
    public void cry() {
        System.out.println("我是猫，我的叫声是喵喵喵");
    }
}

class G {
    public void makeCry(CanCry c) {
        c.cry();
    }

    public static void main(String[] args) {
        Dog dog = new Dog();
        Cat cat = new Cat();
        G g = new G();
        g.makeCry(dog);
        g.makeCry(cat);
    }
}

public class Main {
    public static void main(String[] args) {
        G.main(args);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
我是狗，我的叫声是汪汪汪
我是猫，我的叫声是喵喵喵
```

注意

Java 大小写敏感，定义 makeCry 后不能调用 makecry。

第 13 题 抽象 Shape 与圆、矩形面积

原题编号：11；本书按出现顺序编为第 13 题。

题目

编写 abstract Shape，声明抽象 getArea()；定义 Circle 与 Rectangle 继承 Shape，分别用圆和矩形公式实现 getArea。main 创建两个具体图形并输出面积，展示父类类型数组统一处理不同图形。原题未指定尺寸，本例圆半径 3，矩形长4宽5。

基础思路

Shape 只规定面积方法，由子类分别实现。将两个对象放进 Shape[]，循环时统一调用 getArea。

完整参考代码 · Main.java

```
import java.util.Locale;

abstract class Shape {
    public abstract double getArea();
}

class Circle extends Shape {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    private double length;
    private double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public double getArea() {
        return length * width;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {new Circle(3), new Rectangle(4, 5)};
        String[] names = {"圆面积", "矩形面积"};
        for (int i = 0; i < shapes.length; i++) {
            System.out.printf(Locale.ROOT, "%s：%.2f%n", names[i], shapes[i].getArea());
```

Main.java（续）

```
        }
    }
}
```

样例输入

（无输入，直接运行）

样例输出

```
圆面积：28.27
矩形面积：20.00
```

注意

抽象方法结尾是分号，不写空大括号冒充抽象实现。

第 14 题 员工数组、平均工资项目与应发工资

原题编号：12；本书按出现顺序编为第 14 题。

题目

设计 Employee，字段 name、age、basic、allowance、tax；构造初始化全部字段；total() 返回 basic+allowance-tax。main 使用 Employee[] 保存原表4名员工：孙悟空/22/3210/1240/280，猪八戒/21/1820/980/160，沙和尚/28/2100/1020/210，白骨精/18/2560/1320/245。计算平均 basic、平均 allowance，再输出所有员工基本信息与 total() 结果。按原题把扣税后的结果称为“应发总工资”。

基础思路

用对象数组存4条记录，第一次循环累加基本工资和津贴，除以人数求平均；第二次循环输出每名员工及其工资。

完整参考代码 · Main.java

```
import java.util.Locale;

class Employee {
    private String name;
    private int age;
    private double basic;
    private double allowance;
    private double tax;

    public Employee(String name, int age, double basic, double allowance, double tax) {
        this.name = name;
        this.age = age;
        this.basic = basic;
        this.allowance = allowance;
        this.tax = tax;
    }
```

Main.java（续）

```
    public double getBasic() { return basic; }
    public double getAllowance() { return allowance; }

    public double total() {
        return basic + allowance - tax;
    }

    public void showInfo() {
        System.out.printf(Locale.ROOT,
            "%s 年龄%d 基本工资%.2f 津贴%.2f 扣税%.2f 应发总工资%.2f%n",
            name, age, basic, allowance, tax, total());
    }
}

public class Main {
    public static void main(String[] args) {
        Employee[] employees = {
            new Employee("孙悟空", 22, 3210, 1240, 280),
            new Employee("猪八戒", 21, 1820, 980, 160),
            new Employee("沙和尚", 28, 2100, 1020, 210),
            new Employee("白骨精", 18, 2560, 1320, 245)
        };
        double sumBasic = 0;
        double sumAllowance = 0;
        for (int i = 0; i < employees.length; i++) {
            sumBasic += employees[i].getBasic();
            sumAllowance += employees[i].getAllowance();
        }
        System.out.printf(Locale.ROOT, "平均基本工资：%.2f%n", sumBasic / employees.length);
        System.out.printf(Locale.ROOT, "平均津贴：%.2f%n", sumAllowance / employees.length);
        for (int i = 0; i < employees.length; i++) {
            employees[i].showInfo();
        }
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
平均基本工资：2422.50
平均津贴：1140.00
孙悟空 年龄22 基本工资3210.00 津贴1240.00 扣税280.00 应发总工资4170.00
猪八戒 年龄21 基本工资1820.00 津贴980.00 扣税160.00 应发总工资2640.00
沙和尚 年龄28 基本工资2100.00 津贴1020.00 扣税210.00 应发总工资2910.00
白骨精 年龄18 基本工资2560.00 津贴1320.00 扣税245.00 应发总工资3635.00
```

注意

先求所有人的总和再除以人数，不能在循环中反复除以人数。

第 15 题 Graph 类图：三种图形与 Client 绘制

原题编号：13；本书按出现顺序编为第 15 题。

题目

按原PDF第5页顶部类图实现：Graph 接口声明 public drawing():void；Rectangle 含 private double length、width 和 Rectangle(double length,double width)；Circle 含 private double radius 和 Circle(double radius)；Triangle 含 private double a、b、c 和 Triangle(double a,double b,double c)。三者实现 Graph。Client 包含 public myDrawing(Graph graph):void，根据实际对象调用 drawing。main 创建三种图形并测试；本控制台题以输出形状名称和尺寸表示绘制，原题未要求图形窗口。

基础思路

先把图里的字段和构造逐项转成Java，再让三类实现 drawing；Client 只依赖 Graph，并将绘制工作交给传入对象。

完整参考代码 · Main.java

```
interface Graph {
    void drawing();
}

class Rectangle implements Graph {
    private double length;
    private double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public void drawing() {
        System.out.println("绘制矩形：长=" + length + "，宽=" + width);
    }
}

class Circle implements Graph {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    @Override
    public void drawing() {
        System.out.println("绘制圆：半径=" + radius);
    }
}

class Triangle implements Graph {
    private double a;
    private double b;
    private double c;

    public Triangle(double a, double b, double c) {
        this.a = a;
```

Main.java（续）

```
        this.b = b;
        this.c = c;
    }

    @Override
    public void drawing() {
        System.out.println("绘制三角形：三边=" + a + "、" + b + "、" + c);
    }
}

class Client {
    public void myDrawing(Graph graph) {
        graph.drawing();
    }
}

public class Main {
    public static void main(String[] args) {
        Client client = new Client();
        client.myDrawing(new Rectangle(4, 3));
        client.myDrawing(new Circle(2));
        client.myDrawing(new Triangle(3, 4, 5));
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
绘制矩形：长=4.0，宽=3.0
绘制圆：半径=2.0
绘制三角形：三边=3.0、4.0、5.0
```

注意

必须保留 Triangle 的 a、b、c 和三参数构造，不能只创建空类。

第 16 题 Animal、Ibehavior 与 Lion 类图实现

原题编号：14；本书按出现顺序编为第 16 题。

题目

按原PDF第5页图1实现：Animal 具有 public height:double、weight:Double、bodyColor:String；Ibehavior 接口声明 walking() 和 eating(String food)；Lion 继承 Animal，同时实现 Ibehavior，新增 public toothNumber:int、tailLength:double。测试输出狮子特征，再调用 walking、eating。按图2示例取体重56.0、体高0.6、皮色黄色、牙齿38、尾长1.2；输出“狮子在奔跑！”和“狮子在吃野牛”。

基础思路

Animal 保存共同特征，Lion 新增狮子特征；用 extends Animal implements Ibehavior 同时表达继承和行为实现，打印原图示例。

完整参考代码 · Main.java

```
class Animal {
    public double height;
    public Double weight;
    public String bodyColor;

    public Animal(double height, Double weight, String bodyColor) {
        this.height = height;
        this.weight = weight;
        this.bodyColor = bodyColor;
    }
}

interface Ibehavior {
    void walking();
    void eating(String food);
}

class Lion extends Animal implements Ibehavior {
    public int toothNumber;
    public double tailLength;

    public Lion(double height, Double weight, String bodyColor,
                int toothNumber, double tailLength) {
        super(height, weight, bodyColor);
        this.toothNumber = toothNumber;
        this.tailLength = tailLength;
    }

    public void showInfo() {
        System.out.println("狮子：");
        System.out.println("体重：" + weight + "，体高：" + height + "，皮色："
            + bodyColor + "，牙齿数：" + toothNumber + "，尾巴长度：" + tailLength);
    }

    @Override
    public void walking() {
        System.out.println("狮子在奔跑！");
    }

    @Override
    public void eating(String food) {
        System.out.println("狮子在吃" + food);
    }
}

public class Main {
    public static void main(String[] args) {
        Lion lion = new Lion(0.6, 56.0, "黄色", 38, 1.2);
        lion.showInfo();
        lion.walking();
```

Main.java（续）

```
        lion.eating("野牛");
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
狮子：
体重：56.0，体高：0.6，皮色：黄色，牙齿数：38，尾巴长度：1.2
狮子在奔跑！
狮子在吃野牛
```

注意

不能因为 walking 中文是行走就漏掉原图给出的“狮子在奔跑！”示例。

第 17 题 网络用户类与默认邮箱构造重载

原题编号：15；本书按出现顺序编为第 17 题。

题目

定义网络用户类，保存用户ID、密码、email。建立实例时必须提供ID和密码，email可省略；提供email时原样使用，省略时邮箱为用户ID拼接“@gameschool.com”。Java不支持给普通形参直接写默认值，因此提供三参数构造和二参数重载构造；不提供无参构造。main 分别验证自定义邮箱和默认邮箱。

基础思路

三参数构造保存完整信息，二参数构造调用 this(userId,password,userId+"@gameschool.com")，复用初始化过程。

完整参考代码 · Main.java

```
class NetworkUser {
    private String userId;
    private String password;
    private String email;

    public NetworkUser(String userId, String password, String email) {
        this.userId = userId;
        this.password = password;
        this.email = email;
    }

    public NetworkUser(String userId, String password) {
        this(userId, password, userId + "@gameschool.com");
    }

    public String getUserId() { return userId; }
```

Main.java（续）

```
    public String getEmail() { return email; }

    public void showInfo() {
        System.out.println("用户ID: " + userId + ", 邮箱: " + email);
    }
}

public class Main {
    public static void main(String[] args) {
        NetworkUser u1 = new NetworkUser("zhangsan", "123456", "zhangsan@example.com");
        NetworkUser u2 = new NetworkUser("lisi", "abcdef");
        u1.showInfo();
        u2.showInfo();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
用户ID: zhangsan, 邮箱: zhangsan@example.com
用户ID: lisi, 邮箱: lisi@gameschool.com
```

注意

Java不支持 NetworkUser(String id, String pwd, String email = ...) 这种形参默认值语法。

复习与参考

复习时先做没有输入的题，再做单次输入，最后做循环输入和多个对象的题。遇到相同类名，回到对应题目录单独编译。

原题库：用户提供的阶段一、阶段二题库 PDF。原题中的重复编号、示例编号和歧义已经在封面说明或对应题目中标出。

Oracle Java Tutorials：语言基础

https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html

Oracle Java Tutorials：面向对象概念

https://docs.oracle.com/javase/tutorial/java/concepts/index.html

本文使用 JDK 17 可运行的基础语法；上面的概念教程版本较早，变量、循环、类与继承等基础仍适用。